

Trabalho M2 – Escalonadores

Disciplina: Sistemas Operacionais

Professor: Michael Douglas C A

Autor: Lucas Emanuel Angioletti do Amaral

Entrega: Relatório eletrônico do simulador de tempo discreto com Round Robin e LRU

1. Enunciado do Projeto

O trabalho propõe a implementação de um simulador de tempo discreto, orientado a tiques de relógio, que integra um escalonador de CPU Round Robin com um gerenciador de memória virtual sob demanda. A substituição de páginas deve seguir o algoritmo LRU (Least Recently Used).

Objetivo central: simular, em console, o comportamento de processos que disputam CPU, sofrem page faults e retornam da fila de bloqueados após a penalidade de I/O.

- O relógio começa em tempo 0 e avança de 1 em 1 tique.
- O processo no topo da fila de prontos recebe a CPU.
- A cada tique, o processo tenta acessar a próxima página da sua lista.
- Se a página estiver na RAM, ocorre um hit e o LRU é atualizado.
- Se a página não estiver na RAM, ocorre page fault, o processo bloqueia e a página é carregada.
- Se a RAM estiver cheia, o LRU remove a página menos recentemente usada.

2. Contexto e Explicação da Solução

O simulador foi desenvolvido em C++ e trabalha com uma entrada textual composta por uma linha de configuração seguida das linhas dos processos. A primeira linha informa quantum,

quantidade de frames na RAM e penalidade de I/O. Cada linha seguinte contém o instante de chegada, o identificador do processo e a sequência de páginas acessadas.

A aplicação representa um cenário didático em que o sistema operacional precisa coordenar duas decisões ao mesmo tempo: quem usa a CPU e como a memória principal é ocupada. O Round Robin garante alternância entre processos prontos, enquanto o LRU tenta manter na RAM as páginas mais recentemente utilizadas.

Na prática, com poucos frames e sequências longas de páginas, a simulação tende a produzir muitos page faults, o que é esperado quando há forte pressão de memória e baixo reaproveitamento de páginas.

3. Funcionamento da Implementação

3.1 Estruturas principais

- **Processo:** guarda chegada, identificador, páginas, estado de bloqueio, quantum usado, page faults e tempo de conclusão.
- **Frame:** representa cada espaço físico da RAM e registra a última vez em que foi usado.
- **EventoBloqueio:** controla o instante em que um processo volta da penalidade de I/O.

3.2 Fluxo por tique

- Libera processos que terminaram a penalidade de I/O.
- Insere processos que chegaram ao sistema.
- Entrega CPU ao próximo processo pronto.
- Verifica a próxima página solicitada.
- Em caso de hit, avança a execução e atualiza o LRU.
- Em caso de fault, bloqueia o processo, carrega a página e possivelmente remove outra página via LRU.

3.3 Observação sobre o comportamento corrigido

Durante a validação do projeto, foi identificado que a mesma página poderia ser requisitada repetidamente após o retorno do page fault, o que prolongava artificialmente a simulação. A correção aplicada faz com que, quando a página é carregada na RAM após o fault, o processo avance para a próxima requisição, mantendo o comportamento esperado pelo enunciado.

4. Resultados e Simulações

A execução abaixo foi feita com o arquivo **arquivo_teste.txt**, que no repositório atual contém a configuração: quantum 3, RAM com 2 frames e penalidade de I/O igual a 4.

Resumo estatístico da simulação:

- Total de processos: 20
- Tempo médio de retorno: 311,65 tiques
- Menor tempo de retorno: 280 tiques
- Maior tempo de retorno: 336 tiques
- Média de page faults por processo: 12,40
- Menor quantidade de page faults: 10
- Maior quantidade de page faults: 16

Processo	Tempo de retorno	Total de page faults
P1	296	12
P2	327	14
P3	305	12
P4	284	11
P5	323	13
P6	310	12
P7	323	13
P8	311	12
P9	330	14
P10	293	11
P11	312	12
P12	295	11
P13	323	13
P14	313	12

Processo	Tempo de retorno	Total de page faults
P15	323	13
P16	323	13
P17	324	13
P18	336	16
P19	280	10
P20	302	11

O estado final da RAM ao término da execução foi: frame 0 com a página 4 e frame 1 com a página 5. Isso mostra que, ao final, a memória preservou as páginas mais recentemente utilizadas no encerramento da simulação.

5. Trechos de Códigos Pertinentes da Solução

5.1 Ciclo principal de um tique

```
void executar_tique() {
    liberar_bloqueados();
    adicionar_novas_chegadas();
    despachar_proximo();

    if (processo_ativo == -1) {
        registrar_log("[Tempo " + to_string(tempo_atual) + "] CPU ociosa.");
        tempo_atual++;
        return;
    }

    int indice_processo = processo_ativo;
    Processo& processo = processos[indice_processo];
    ...
}
```

5.2 Tratamento de page fault

```
void bloquear_processso_por_page_fault(int indice_processo, int pagina) {
    Processo& processo = processos[indice_processo];
    processo.bloqueado = true;
    processo.desbloqueio_em = tempo_atual + penalidade_io;
    processo.page_faults++;
}
```

```

        processo.quantum_usado = 0;
        fila_bloqueados.push({processo.desbloqueio_em, processo.ordem_bloqueio,
        indice_processo});
        processo_ativo = -1;

        carregar_pagina_na_ram(pagina);
        processo.proxima_pagina++;
    }

```

5.3 Escolha da vítima pelo LRU

```

int escolher_frame_lru() const {
    int escolhido = -1;
    long long menor_uso = LLONG_MAX;
    for (int i = 0; i < static_cast<int>(ram.size()); ++i) {
        if (!ram[i].ocupado) {
            return i;
        }
        if (ram[i].ultimo_uso < menor_uso ||
            (ram[i].ultimo_uso == menor_uso && (escolhido == -1 || i < escolhido))) {
            menor_uso = ram[i].ultimo_uso;
            escolhido = i;
        }
    }
    return escolhido;
}

```

6. Análise e Discussão

Os resultados mostram um cenário típico de alta pressão sobre a memória: a RAM possui apenas dois frames, enquanto os processos usam sequências longas e intercaladas de páginas. Isso faz com que o LRU seja acionado com frequência e que muitos acessos acabem convertidos em page faults.

O escalonador Round Robin garante justiça na disputa pela CPU, mas, com a penalidade de I/O e a limitação de memória, vários processos passam períodos bloqueados. Isso explica os intervalos com CPU ociosa observados durante a execução.

Como conclusão técnica, o simulador atende ao objetivo pedagógico do trabalho: ele expõe, em tempo discreto, a interação entre escalonamento de CPU, bloqueio por page fault e política de substituição de páginas. A combinação RR + LRU produz um comportamento coerente com sistemas operacionais reais, especialmente em cenários de pouca memória disponível.

7. Conclusão

O projeto implementa com sucesso um simulador de tempo discreto com Round Robin e LRU, apresentando logs de execução, tempo de retorno por processo, total de page faults e estado final da RAM. O arquivo HTML foi preparado para ser exportado diretamente em PDF, atendendo ao formato solicitado no trabalho.
